L&T EduTech

Project Statement 6

AI-Ready Data Engineering Pipeline for Predictive Maintenance

Final Technical Documentation

Submitted by	Ujjval
Roll No. / Batch	L&T EduTech Project Statement 6
Programme	AI-Ready Data Engineering Pipeline for Predictive Maintenance
Mentor / Evaluator	L&T EduTech Evaluation Committee
Submission date	November 2026
GitHub Repository	https://github.com/UjjvxL/AI-Ready-Predictive-Maintenance-Pipeline

Deliverables covered: Data Pipeline | Cleaned Dataset | Feature Engineering Report | Dashboard | Final Technical Documentation

Contents

1. Executive Summary
2. Problem Understanding and Approach
3. Solution Architecture and Tool Choices
4. Data Sources and Dataset Description
5. Data Ingestion and Integration
6. Data Cleaning and Validation
7. Handling Missing Values
8. Feature Engineering Report
9. The AI-Ready Dataset and How It Was Validated
10. Dashboard and Insights
11. Limitations, Risks and Future Work
12. Reproducibility: Project Structure and How to Run
13. Conclusion and Learning Outcomes

Appendix A. Feature Catalogue (data dictionary)

Appendix B. Power BI Build Guide and DAX Measures

1. Executive Summary

Predictive-maintenance AI is only as good as the data under it. Raw machine data is rarely usable as delivered: sensors drop out, units differ between machines, timestamps arrive in several formats, and failures are recorded in a different system from the readings. This project builds a reproducible data engineering pipeline that turns that raw material into a trustworthy, model-ready dataset, and shows through a dashboard what the cleaned data reveals.

What was built. An end-to-end Python pipeline (Pandas, NumPy, scikit-learn) that ingests three sources, integrates them, validates and cleans the readings, imputes missing values in two stages, builds 79 candidate features (6 raw sensors plus 73 engineered), prunes them to 59, creates two prediction targets, applies a leakage-safe time split and scaling, and exports the results. A dashboard (two pages, plus a Power BI-ready star schema and build guide) presents data quality and machine health.

Headline results (all produced by running the code in this submission):

- **57,311 raw sensor rows** from 20 machines over 120 days were reduced to **56,744 validated hourly rows**, with **0 missing values, 0 duplicates and 0 out-of-range readings** after cleaning (enforced by assertions that stop the pipeline if violated).
- The pipeline found and fixed **22,280 data defects** (excluding missing values) across seven categories, including 3 machines reporting temperature in Fahrenheit.
- **12,060 values (3.5% of all sensor cells)** were imputed; every one is flagged in the dataset so models can treat them with appropriate caution. The imputation method was chosen by a hide-and-recover experiment, not by assumption.
- The final **AI-ready dataset has 54,784 rows and 59 features**, with RUL (remaining useful life) and failure-within-72-hours targets.
- As a proof of readiness, baseline models trained on it detect upcoming failure with F1 = 0.985 (engineered features) versus 0.950 (raw sensors only), and predict RUL with a mean error of 20.8 hours versus 65.9 hours for a naive guess.

Important honesty note. The project statement does not supply a dataset, so the data used here is **simulated** by `src/generate_data.py` (fixed random seed) to behave like real plant data, including deliberately injected defects. The pipeline code is dataset-agnostic, but the very high model scores should be read as proof that the pipeline preserves a learnable signal, **not** as a prediction of performance on real machines, where degradation is noisier. See Section 11.

2. Problem Understanding and Approach

2.1 What the assignment asks

The statement (Project Statement 6) is: "Industries require high-quality datasets before applying Artificial Intelligence for predictive maintenance. Design a data engineering solution that prepares machine sensor data for AI applications." The emphasis is on **data preparation**, not on building the best prediction model. A good solution therefore has to show how raw data becomes reliable, well-described and model-ready, and to prove that it did.

2.2 Why this matters

Predictive maintenance estimates when a machine will fail so that repairs can be scheduled before breakdown rather than after. Models learn from historical sensor behaviour (vibration, temperature, pressure and so on) leading up to past failures. If the history contains unit mix-ups, frozen sensors or gaps, the model learns noise or, worse, a false pattern. Data engineering is the step that prevents this.

2.3 Requirement traceability

Each objective and deliverable in the statement is mapped to what was built and where it is documented.

Table 1. Requirements traceability matrix

Requirement (from statement)	How it is addressed	Where
Collect and integrate machine data	3 heterogeneous sources ingested, keys standardised, mixed timestamps parsed, joined to asset and maintenance data; SQLite warehouse (MongoDB module provided)	Sec. 5
Perform data cleaning and validation	Rule-based validation: schema, ranges, units, duplicates, flat-lined sensors, statistical spikes; post-clean assertions	Sec. 6
Handle missing values	Two-stage imputation selected by a hide-and-recover experiment; every imputed cell flagged	Sec. 7
Apply feature engineering techniques	79 candidate features in 9 families (6 raw + 73 engineered); correlation pruning to 59; importance analysis; leakage controls	Sec. 8
Prepare an AI-ready dataset and visualise insights	Scaled and unscaled datasets with time-based split; baseline-model validation; 2-page dashboard	Sec. 9-10
Deliverable: Data Pipeline	`src/pipeline.py` (+ generator, model check, dashboard, Mongo module)	Sec. 12
Deliverable: Cleaned Dataset	`cleaned_sensor_data.csv` and `ai_ready_dataset*.csv`	Sec. 6, 9
Deliverable: Feature Engineering Report	Section 8 and Appendix A (full data dictionary)	Sec. 8
Deliverable: Dashboard	Dashboard pages, Power BI star-schema exports, DAX and build guide	Sec. 10, App. B
Deliverable: Final Technical Documentation	This document	-

2.4 Scope and assumptions

- Hourly sensor data for 20 machines (pumps, compressors, motors, turbines) over 120 days (1 Jan to 30 Apr 2025), six sensors each.
- Failure is predicted at two levels: a binary label (failure within the next 72 hours) and a numeric RUL in hours, capped at 336 h (14 days).
- A 72-hour horizon is assumed to be a realistic maintenance-planning window; it is a single constant in the code and can be changed.

- Time order is respected everywhere: features use only past data, and train/test are split by date, never shuffled.

3. Solution Architecture and Tool Choices

The pipeline is a sequence of small, testable stages. Raw files are never modified; every stage writes its output and its statistics to disk so the work is auditable.

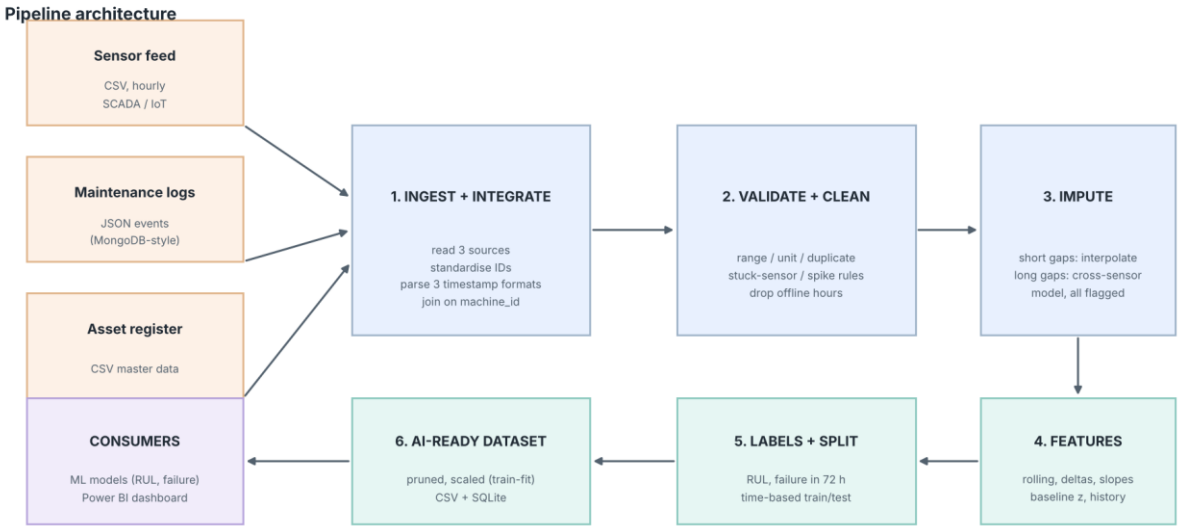


Figure 1. End-to-end pipeline architecture

Table 2. Recommended tools versus what was actually used

Tool	Role in the solution	Status in this submission
Python 3	Orchestration of all stages	Used
Pandas / NumPy	Ingestion, integration, cleaning, rolling-window features, labels	Used
scikit-learn	Cross-sensor imputation (IterativeImputer), scaling, baseline models	Used
SQLite	Executed storage layer (`data/warehouse.db`) with dimension and fact tables	Used (substitute for MongoDB)
MongoDB	Document store for raw readings and semi-structured maintenance logs; time-series collections	Code provided (`mongo_store.py`), **not executed**: no database server or pymongo in the build environment
Matplotlib	Dashboard pages and report figures	Used (substitute for the rendering step in Power BI)
Power BI	Interactive dashboard	Star-schema CSVs, DAX and build guide provided; the .pbix file must be assembled in Power BI Desktop (Appendix B)

3.1 Design principles

- **Fail loudly.** Post-clean assertions stop the pipeline if any NaN, duplicate key, unknown machine or out-of-range value survives.
- **Never silently invent data.** Every imputed value carries a flag (0 = observed, 1 = interpolated, 2 = model-estimated).
- **Choose methods by evidence.** Imputation was selected by measured error on hidden values, and features were pruned by measured correlation.

- **No leakage.** Features use only past information; the scaler is fitted on training rows only; the split is chronological with a 72-hour gap.
- **Reproducible.** Fixed seeds, a configuration block at the top of `pipeline.py`, and a log file for every run.

4. Data Sources and Dataset Description

Three sources mirror what a real plant would hold in separate systems. They were generated with a fixed seed so every run reproduces the same data.

Table 3. Raw data sources

Source	Format	Content	Volume
sensor_readings.csv	CSV	Hourly readings per machine: temperature, vibration, pressure, rpm, current, oil quality	57,311 rows
maintenance_logs.json	JSON (semi-structured)	Work orders: failure and preventive events with failure mode, downtime hours, cost (INR)	95 events (37 failures, 58 preventive)
machine_master.csv	CSV	Asset register: type, location, install date, rated rpm, rated power	20 machines

Table 4. Sensor dictionary and validation ranges

Sensor	Unit	Plausible range used for validation	Physical meaning
temperature	deg C	0 to 150	Bearing / casing temperature; rises as friction or load increases
vibration	mm/s RMS	0 to 25	RMS vibration velocity; the classic early indicator of imbalance and bearing wear
pressure	bar	0.1 to 15	Process pressure; falls with seal leaks and internal wear
rpm	rev/min	200 to 3600	Shaft speed; slips under mechanical drag
current	A	0 to 150	Motor current draw; rises with mechanical resistance
oil_quality	index 0-100	0 to 100	Lubricant condition index; degrades as wear particles accumulate

4.1 Deliberately injected data-quality problems

To give the pipeline realistic work to do, the generator injects the defects that are most common in industrial telemetry. This also lets the results be checked: the detectors should find roughly what was injected.

Table 5. Injected defects

Defect	How it was injected	Why it matters for AI
Missing values	~3% random per sensor, plus 25 multi-hour sensor-dropout blocks	Most models cannot accept NaN; naive row deletion discards data
Unit inconsistency	Temperature reported in deg F for 3 machines	A model would see 160 'degrees' and 70 'degrees' as the same quantity
Spikes / outliers	~0.4% of each sensor multiplied by a large factor	Distort rolling statistics and scaling
Impossible values	rpm = -999, pressure = -1, oil quality = 180	Sentinel codes silently corrupt averages
Flat-lined sensor	Vibration on one machine frozen for 3 days	Looks valid but carries no information, hiding a developing fault
Duplicates	~1% of rows repeated	Over-weights some observations; breaks unique keys
Mixed timestamps	Three different formats (including day-first)	Day/month confusion silently misorders the time series

Defect	How it was injected	Why it matters for AI
Messy labels	Lower-case / padded machine IDs; inconsistent type and location spellings	Breaks joins and groupings

5. Data Ingestion and Integration

5.1 Ingestion

The three files are read with explicit types: `machine_id` and `timestamp` are read as text so that nothing is silently reinterpreted on load. The maintenance JSON is flattened into a table. No raw file is ever modified.

5.2 Integration steps

- **Key standardisation.** Machine IDs are trimmed and upper-cased. 1,124 rows had a non-standard ID; after standardisation all 56,744 surviving rows match the asset register.
- **Timestamp parsing.** Three formats were present: ISO with space (46,941 rows), day-first with slashes (6,928) and ISO with a trailing Z (3,442). Each format is matched by a regular expression and parsed with an explicit format string, never with a guessing parser, because 03/04 could be 3 April or 4 March. Unparseable rows: 0.
- **Master-data repair.** Machine type and location labels were normalised (e.g. 'pump ' and 'MOTOR' became 'Pump' and 'Motor'); one missing rated rpm was repaired from the machine's own telemetry (nearest standard rating).
- **Regular time grid.** Each machine is re-indexed to a complete hourly grid so gaps become explicit. Gaps that fall inside a recorded maintenance downtime window are classified as genuine **offline hours** and removed; 856 such hours were removed. Gaps that are *not* explained by downtime would be classed as telemetry loss and imputed (count in this dataset: 0).
- **Joins.** Sensor rows are joined to the asset register on machine_id (type, age, rating) and to maintenance history by timestamp using only events that have already happened.

5.3 Storage layer

The executed pipeline writes a SQLite warehouse (`data/warehouse.db`) with four tables: `dim_machine`, `fact_maintenance`, `fact_sensor_clean` (indexed on machine and timestamp) and `fact_labels`. This gives a queryable, schema-enforced store for the dashboard layer.

The recommended tool list names MongoDB, which suits this data well: maintenance logs are semi-structured events whose fields vary by event type, and sensor data fits MongoDB time-series collections (timeField = timestamp, metaField = machine_id). `src/mongo_store.py` implements this design, including indexes and an example aggregation. **It was not executed here** because neither a MongoDB server nor the pymongo package was available, so it has not been tested; run it against a local MongoDB to verify before relying on it.

```
# Example MongoDB document (maintenance_logs)
{ "machine_id": "M001", "event_ts": "2025-02-14 06:00:00", "event_type": "FAILURE",
  "failure_mode": "Bearing wear", "downtime_hours": 18.5, "cost_inr": 212000 }

# Time-series collection definition
db.create_collection("sensor_readings_clean",
    timeseries={"timeField": "timestamp", "metaField": "machine_id", "granularity": "hours"})
```

6. Data Cleaning and Validation

Validation is rule-based and ordered. Each rule records how many values it touched, so the effect of every step is visible and auditable.

Table 6. Cleaning and validation rules, in execution order

Step	Rule	Action	Result
1	Machine ID must match asset register after trim and upper-case	Standardise	1,124 rows fixed; 0 unknown
2	Timestamp must parse under one of three known formats	Explicit parse, round to hour	0 unparseable
3	(machine, hour) must be unique	Drop repeats, keep first	567 duplicates removed
4	Machine median temperature above 110 means deg F	Convert to deg C	M004, M011, M017: 8,533 rows converted
5	Value must lie inside the physical range (Table 4)	Set to missing	624 values
6	10+ identical consecutive readings means a stuck sensor	Set to missing	72 values (vibration)
7	Hampel filter (13-hour window, 6 MADs) with a noise floor of 5x typical hourly change	Set to missing	990 values
8	Hours inside recorded downtime are offline, not missing	Remove row	856 hours

6.1 Per-sensor detections

Table 7. Defects detected per sensor

Sensor	Out of range	Stuck	Spikes	Missing (raw)	Missing %
temperature	215	0	15	1,775	3.1%
vibration	9	72	215	1,705	3.0%
pressure	80	0	217	1,775	3.1%
rpm	254	0	101	1,705	3.0%
current	6	0	213	1,745	3.0%
oil_quality	60	0	229	1,770	3.1%

The spike counts (roughly 215 to 230 per sensor) are consistent with the ~0.4% injection rate. For temperature, most injected spikes exceeded the 150 deg C range limit and were caught by rule 5 first, which is why its spike count is low. Early in development, an unfloored Hampel filter flagged several hundred genuine readings per sensor as outliers because short windows occasionally have an unusually small MAD; adding the noise floor in rule 7 fixed this. Likewise, an initial oil-quality range check wrongly flagged healthy readings near the ceiling of 100, which was traced to the simulator and corrected at the source. Both are examples of why detector output should be sanity-checked against expectations rather than trusted blindly.

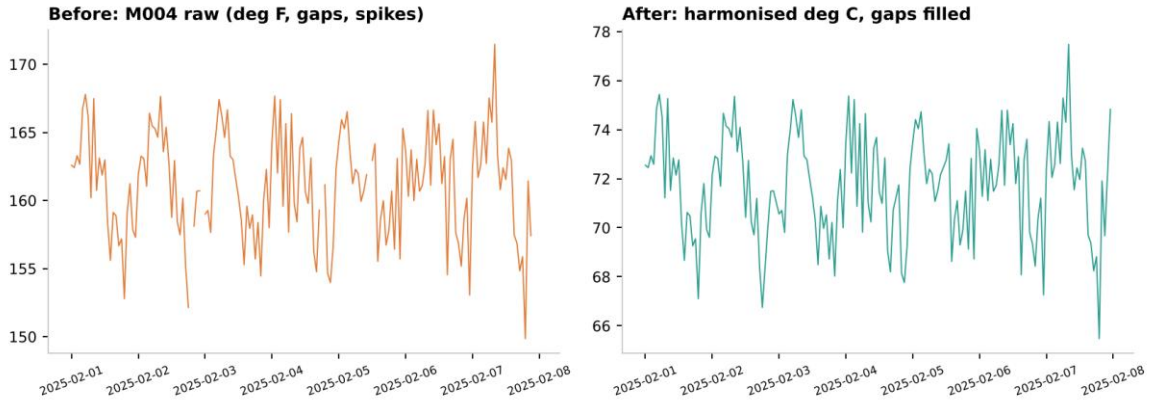


Figure 2. Machine M004 temperature for one week, before (raw, deg F, with gaps and spikes) and after cleaning (deg C, gaps filled)

6.2 Data-quality scorecard

Table 8. Data-quality scorecard before and after

Dimension	Definition	Raw	Cleaned
Completeness	Share of sensor cells that are non-missing	96.95%	100%
Validity	Share of cells free of range violations, spikes, stuck values and unit errors	97.03%	100%
Uniqueness	Share of rows that are not duplicates on (machine, hour)	99.01%	100%
Consistency	Share of ID and timestamp fields already in canonical form	89.97%	100%

The cleaned figures of 100% are guaranteed by the post-clean assertions, not merely observed: the pipeline cannot finish if they are violated. The raw figures use the definitions above and count each defective cell once per category, so they are indicative rather than a single formal standard.

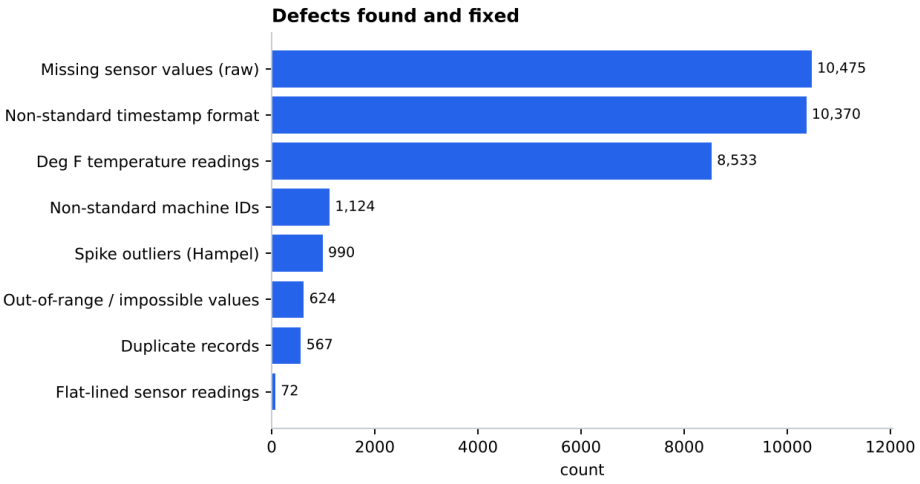


Figure 3. Defects found and fixed, by category

7. Handling Missing Values

7.1 Why missing data needs thought

After validation, **12,060 sensor values were missing**: those missing in the raw feed plus the values the validation rules deliberately blanked (impossible readings, spikes and stuck values). Simply deleting these rows would discard the other five good sensors in each row and tear holes in the time series that break rolling

features. Imputing blindly with the mean would flatten the very degradation trends we want to detect. The right method depends on the gap, so it was tested rather than assumed.

7.2 Experiment: hide known values, recover them, measure the error

About 1,500 known temperature and vibration values were hidden in two scenarios: scattered single points, and 12-hour blocks (a sensor dropout). Five methods then tried to recover them; error is measured against the true value.

Table 9. Mean absolute error of each imputation method (best in bold; temperature in deg C, vibration in mm/s)

Sensor	Gap type	Global mean	Machine median	Forward fill	Linear interp.	Cross-sensor model
temperature	single points	3.377	3.080	1.707	1.486	**1.397**
temperature	12-hour blocks	3.813	3.593	2.440	1.888	**1.387**
vibration	single points	0.677	0.589	0.288	**0.258**	0.290
vibration	12-hour blocks	0.751	0.658	0.333	0.313	**0.289**

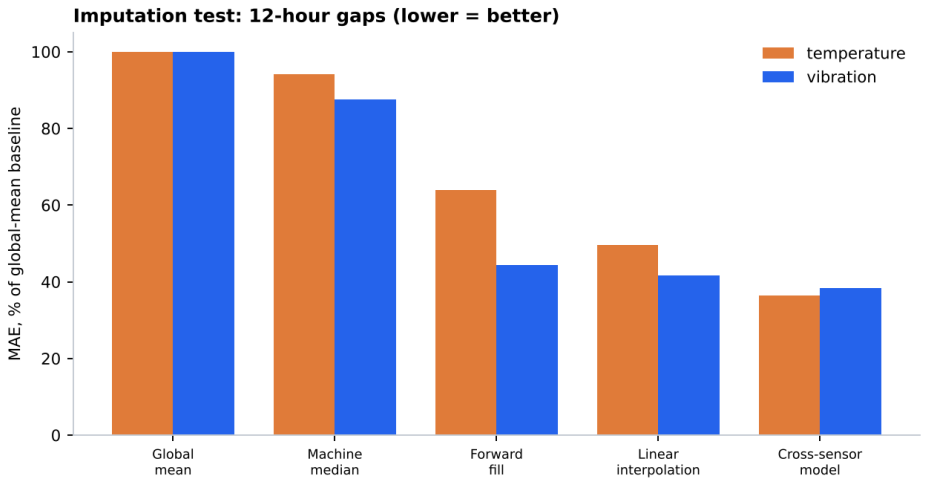


Figure 4. Imputation error for 12-hour gaps, relative to the global-mean baseline (lower is better)

7.3 Findings and the strategy chosen

- Mean and median fills are the worst by a wide margin, confirming that they should not be used on trending sensor data.
- For **single missing points**, linear interpolation and the cross-sensor model are close (the difference is small and the winner depends on the sensor).
- For **12-hour blocks**, the cross-sensor model is clearly better for temperature (MAE 1.39 versus 1.89 deg C for interpolation), because interpolation can only draw a straight line across a gap while the model uses what the other five sensors were doing during it.
- Forward fill is competitive on vibration but systematically lags behind a trend.

Chosen strategy (two stages):

- **Stage 1: linear interpolation for gaps up to 6 hours**, applied only inside a continuous run of data (never across a downtime boundary). It is simple, transparent and accurate for short gaps. Cells filled: 11,835.
- **Stage 2: cross-sensor model for everything else** (longer gaps and the edges of segments), using scikit-learn's IterativeImputer with a Bayesian ridge regressor, which predicts each missing sensor from the others. Cells filled: 225. This also covers the 72-hour stuck vibration sensor, where interpolation would have drawn a meaningless straight line.
- Results are clipped to the physical range, and each cell receives a flag (1 = interpolated, 2 = model-estimated) in the cleaned dataset.

Table 10. Imputation summary

Sensor	Missing before	Interpolated	Model-estimated	Missing after
temperature	1,991	1,965	26	0
vibration	1,986	1,881	105	0
pressure	2,054	2,040	14	0
rpm	2,042	2,015	27	0
current	1,947	1,918	29	0
oil_quality	2,040	2,016	24	0
Total	**12,060**	**11,835**	**225**	**0**

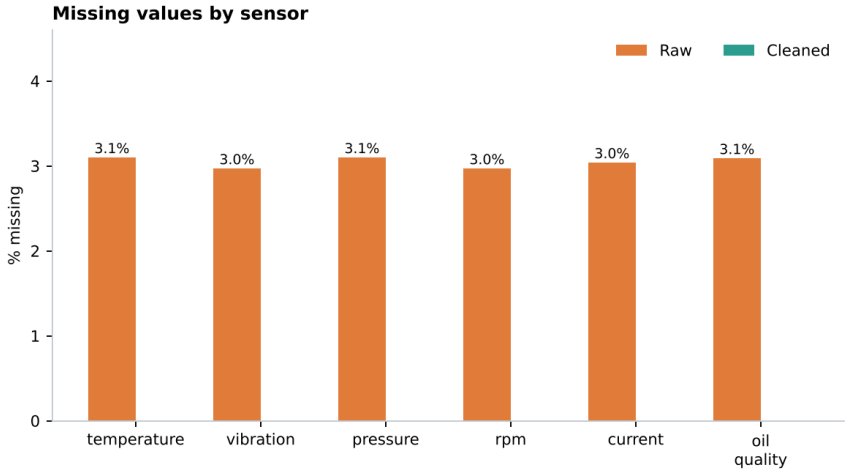


Figure 5. Missing values by sensor, raw versus cleaned

Limitation. Imputed values are estimates. They make up 3.5% of sensor cells, and the evaluation above was run on the genuinely observed data, so the quoted errors are realistic for similar gaps. Because imputed cells are flagged, a downstream user can exclude them or test whether model results depend on them.

8. Feature Engineering Report

Raw instantaneous readings say little about whether a machine is degrading. A temperature of 85 deg C may be normal for one machine under load and alarming for another at idle. Feature engineering expresses each reading in context: its recent history, its direction of change, its distance from the machine's own healthy baseline, and what has been done to the machine.

8.1 Feature families

Table 11. Feature families

Family	What it contains	Why it helps	Built	Kept
Raw sensor values	Cleaned hourly readings (the baseline for comparison)	Reference level; what a model without engineering would see	6	6
Rolling statistics	Mean, standard deviation, max and min over the last 6 h and 24 h	Smooths noise; max/min capture intermittent excursions; standard deviation captures instability	36	23
Rate of change	Difference versus 1 h and 6 h earlier	Fast-developing faults show up as a change before the level looks abnormal	12	12
Trend	24 h linear-regression slope and 12 h exponentially weighted mean (temperature, vibration, oil quality)	Degradation is a direction, not a level	5	3

Family	What it contains	Why it helps	Built	Kept
Baseline deviation	Robust z-score of the 6 h mean against the machine's own first-14-day healthy baseline	Makes machines comparable: each is measured against itself	6	3
Cross-sensor ratios	Vibration per amp; temperature-to-pressure ratio	Faults often change the relationship between sensors before any single one is extreme	2	0
Asset context	Rated speed ratio, rated power, machine age, machine type (one-hot)	Lets one model serve different machine types and ages	7	7
Time of day / week	Hour as sine/cosine, day of week	Captures daily load cycles without a false jump at midnight	3	3
Maintenance history	Hours since last maintenance event, count of failures to date (past events only)	Recently repaired machines behave differently from long-running ones	2	2
Total			**79**	**59**

8.2 Leakage controls

Leakage, where a feature secretly contains information from the future, is the commonest cause of models that look excellent in testing and fail in production. The following controls are built in:

- All rolling and lag features look backwards only.
- Maintenance-history features use only events with a timestamp at or before the row (a binary search on event times).
- The healthy baseline uses only the first 14 days of each machine's history and a robust median/MAD, so it is not pulled by later failures.
- Targets (RUL, failure within 72 h) are computed from future failure events and are kept out of the feature set.
- The final 72 hours of each machine's record, where no later failure exists, are dropped (1,384 rows): it is impossible to know whether a failure would have followed.
- The scaler is fitted on training rows only; the train/test split is chronological with a 3-day gap so that no label window straddles the boundary.

8.3 Target variables

- **fail_within_72h** (classification): 1 if the next failure occurs within 72 hours, else 0.
- **RUL_capped** (regression): hours until the next failure, capped at 336 h. Rows with no later failure are assigned the cap (a standard treatment for right-censored data).

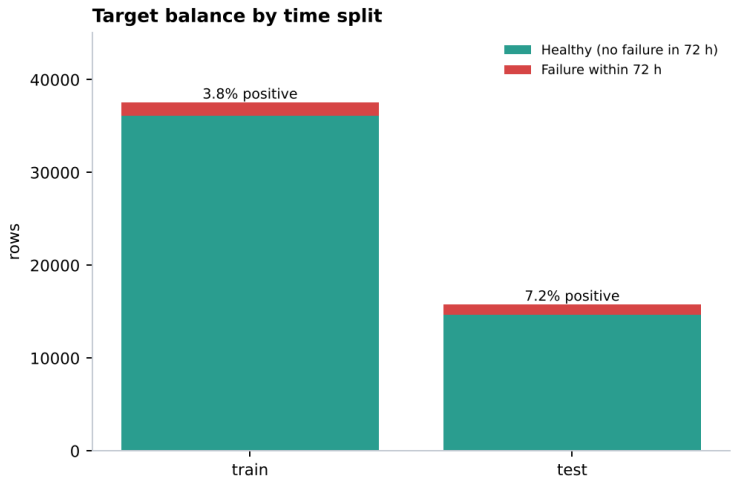


Figure 6. Target balance after the time-based split

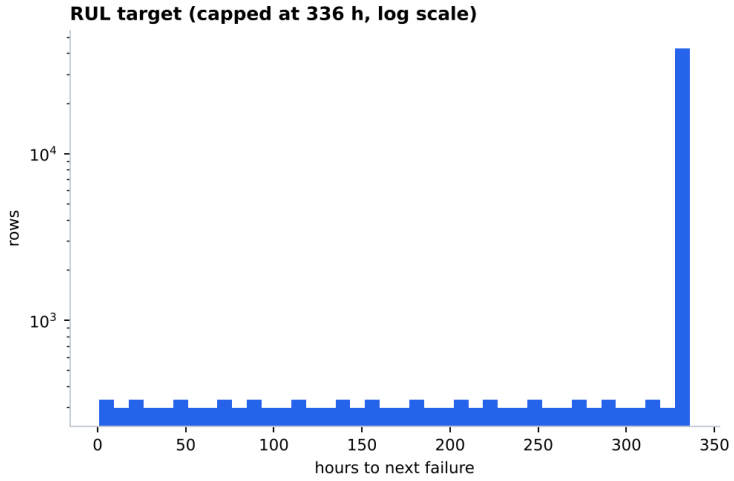


Figure 7. Distribution of the RUL target

8.4 Feature selection

Rolling windows overlap heavily, so many features are near-duplicates. Pairwise Pearson correlation was computed and, scanning in order of interpretability, any feature with absolute correlation above 0.98 with an already-kept feature was removed. This reduced the set from 79 to **59** features (20 removed). No feature had zero variance.

Table 12. Examples of pruned features (full list in Appendix A)

Removed feature	Redundant with	Abs. correlation
temperature_min_24h	temperature_mean_24h	0.982
vibration_mean_6h	temperature_mean_24h	0.985
vibration_mean_24h	temperature_mean_24h	0.994
vibration_min_24h	temperature_mean_24h	0.982
pressure_mean_24h	temperature_mean_24h	0.991
rpm_mean_6h	rpm	1.000
rpm_mean_24h	rpm	1.000
rpm_max_24h	rpm	0.999

A caution worth stating: in this simulated data, the sensors move together as a machine degrades, so some pruned features are correlated with others only because of the shared degradation process. On real data the correlation structure will differ and the pruning threshold should be revisited.

Table 13. Top 10 features by random-forest importance

Rank	Feature	Family	Importance
1	temperature_mean_24h	Rolling statistics	0.127
2	vibration_max_24h	Rolling statistics	0.112
3	vibration	Raw sensor values	0.101
4	temperature_mean_6h	Rolling statistics	0.099
5	temperature_max_24h	Rolling statistics	0.077
6	pressure_mean_6h	Rolling statistics	0.073
7	pressure_min_24h	Rolling statistics	0.056
8	pressure_max_24h	Rolling statistics	0.044
9	oil_quality	Raw sensor values	0.044
10	temperature	Raw sensor values	0.039

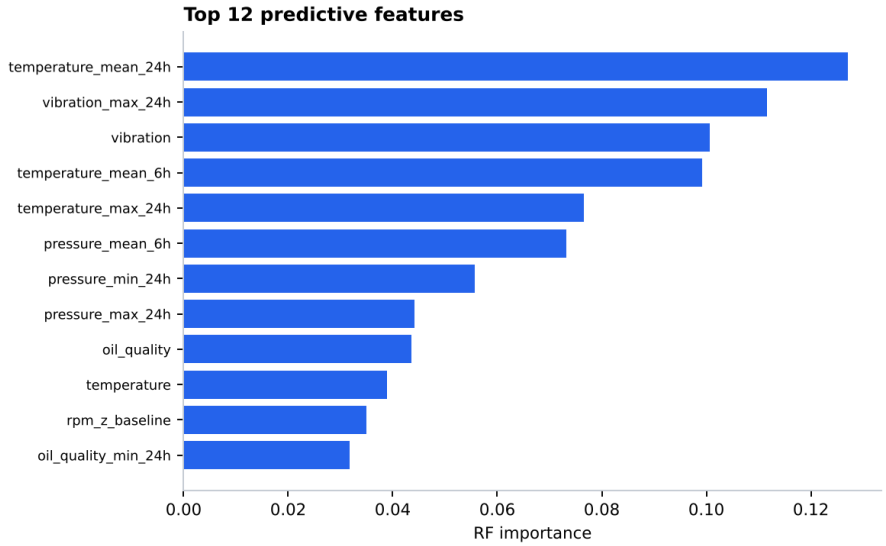


Figure 8. Top predictive features

Engineered features dominate the ranking: of the ten most important, 3 are raw readings and the other 7 are engineered. Importance values from a random forest can be split among correlated features, so the ranking shows which signals matter, not a precise contribution of each.

9. The AI-Ready Dataset and How It Was Validated

9.1 Specification

Table 14. AI-ready dataset specification

Property	Value
Rows	54,784 hourly observations across 20 machines
Features	59 (see Appendix A)
Targets	fail_within_72h (binary), RUL_capped (hours), RUL_hours (uncapped, NaN when no later failure)
Split	Train 37,561 rows (to 21 Mar 2025); gap 1,440 rows (22-24 Mar, excluded); test 15,783 rows (from 25 Mar 2025)
Positive rate	Train 3.8% (1,440 rows); test 7.2% (1,144 rows)
Scaling	StandardScaler fitted on training rows only; one-hot columns left unscaled; parameters saved in `scaler_params.json`
Files	`ai_ready_dataset.csv` (unscaled, for tree models and inspection); `ai_ready_dataset_scaled.csv` (for distance- or gradient-based models)
Rows dropped on the way	856 offline hours; 1,384 censored end-of-record rows; 576 warm-up/incomplete-window rows

9.2 Proof of readiness: baseline models

A dataset is AI-ready if models can learn from it without errors or leakage. Two random-forest baselines were trained on the training period and evaluated on the unseen test period: one on the six raw sensors only, one on the engineered features. As a second check, a 5-fold evaluation held out entire machines (so the model is tested on machines it has never seen).

Table 15. Failure-within-72h baseline results

Metric (test period)	Raw sensors only	Engineered features
Features used	6	59
ROC-AUC	0.9996	0.9999

Metric (test period)	Raw sensors only	Engineered features
PR-AUC	0.9953	0.9992
Precision (threshold 0.5)	0.917	0.974
Recall (threshold 0.5)	0.986	0.996
F1	0.950	0.985
Leave-machines-out AUC (5-fold)	0.9996 +/- 0.0004	1.0000 +/- 0.0000

For RUL, the engineered-feature model's mean absolute error on the test period is **20.8 hours**, compared with **65.9 hours** for always predicting the training average.

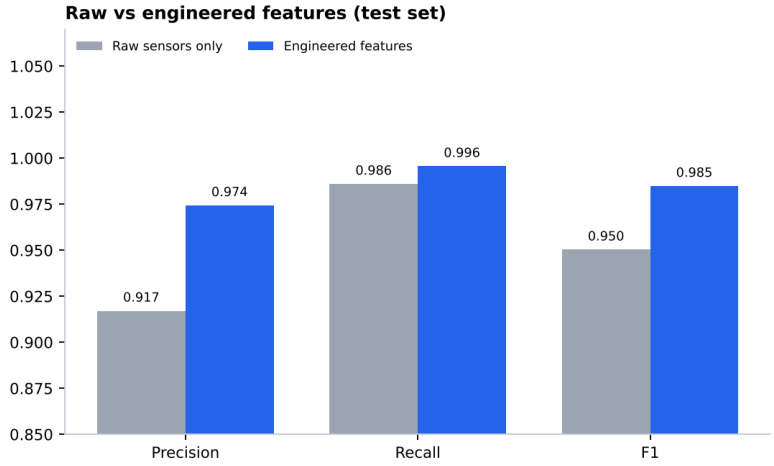


Figure 9. Precision, recall and F1 on the test period, raw versus engineered features

How to read these numbers. Scores this high are a property of the simulated data, in which degradation is smooth and strongly visible in several sensors. They demonstrate that the pipeline preserves a learnable signal and that the dataset is free of structural errors; they do **not** predict performance on a real plant. The ROC-AUC is saturated for both models, so the more informative comparison is precision and F1: engineering the features lifts precision from 0.917 to 0.974 (fewer false alarms) and recall from 0.986 to 0.996. The test period contains only 1,144 positive rows from a handful of failures, so the estimates are noisy.

10. Dashboard and Insights

The dashboard has two pages. Page 1 shows what was wrong with the data and what the pipeline did about it. Page 2 shows machine health and the readiness of the dataset for AI. The pages were rendered from the pipeline outputs with Matplotlib, and the same views can be built in Power BI from the exported star schema (Appendix B).

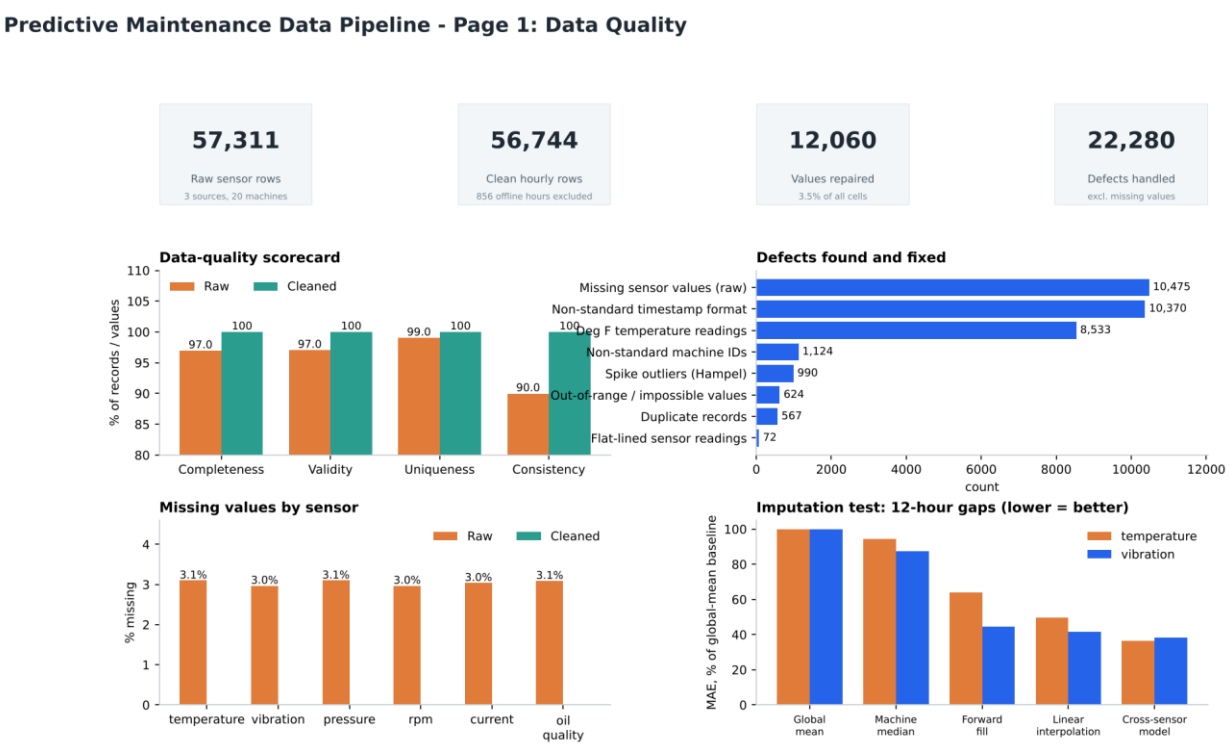


Figure 10. Dashboard page 1: data quality



Figure 11. Dashboard page 2: machine health and AI readiness

10.1 What the data shows

- **Failures leave a clear signature.** In the 72 hours before a failure, vibration averages about 7.9 mm/s versus 3.0 mm/s on healthy machines (+158%), temperature is up 30%, current up 13%, pressure down 19% and oil quality down 16%. Vibration shows the largest relative change of the six sensors.
- **The fleet heatmap (Figure 12) makes failures visible as bursts.** Each hot cell is a week in which a machine's vibration ran well above its own baseline. All 41 machine-weeks above 1.5x baseline had a failure in that week or the following 7 days, compared with about 10% of the other machine-weeks (a pattern that is partly built into the simulator, so it shows the pipeline preserves the signal rather than proving it would hold on real plants).
- **The log contains 37 failures** causing 841 hours of downtime and INR 10,971,971 in repair cost. The most frequent failure mode was imbalance (13 events), then electrical fault (7) and bearing wear (7).
- **Fleet mean time between failures** is about 1,557 operating hours per failure (20 machines x 2,880 hours / 37 failures, ignoring downtime).
- **Preventive events** (58) do not remove telemetry in this data, so they appear in the maintenance-history features but not as offline gaps.

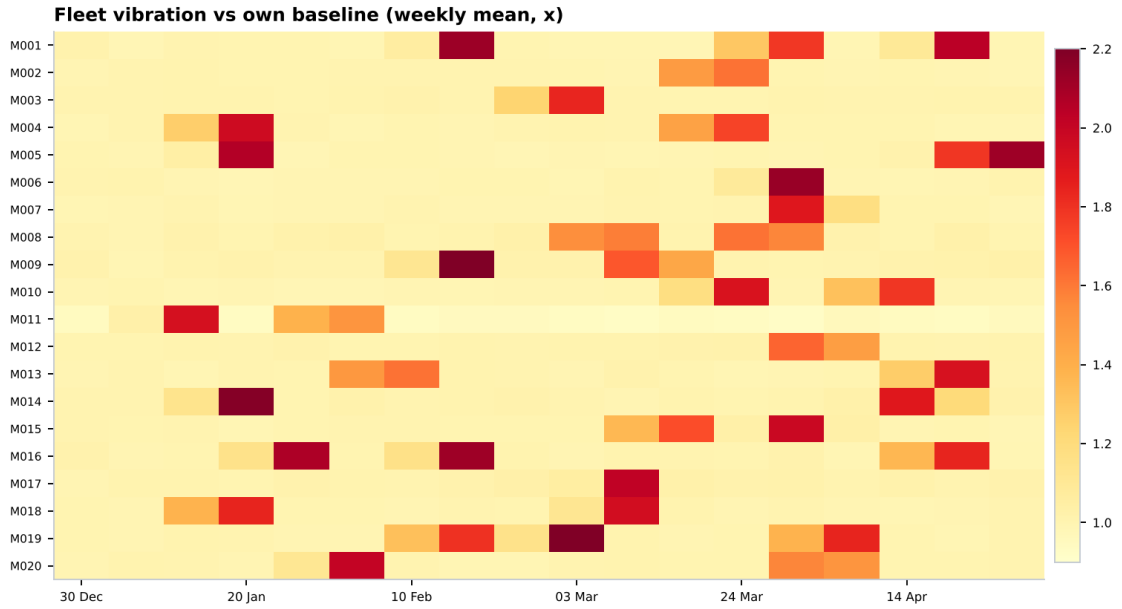


Figure 12. Weekly mean vibration relative to each machine's own first-14-day baseline

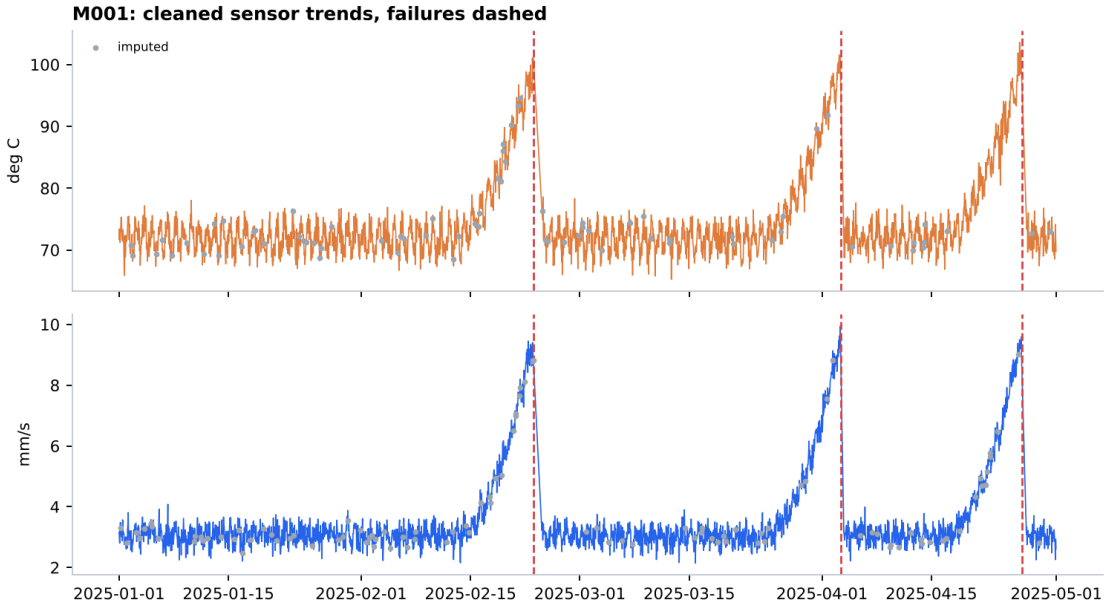


Figure 13. Cleaned temperature and vibration for machine M001 (the machine with the most failures); dashed lines mark failures, grey dots mark imputed values

In Figure 13, vibration and temperature climb ahead of each failure (the simulator builds in a roughly ten-day ramp) and reset after repair. Imputed points (grey) sit on the same curve as observed data and do not distort the pattern.

11. Limitations, Risks and Future Work

11.1 Limitations (stated plainly)

- **The data is simulated.** All results demonstrate the pipeline and its behaviour on realistic defects, not real-plant performance. Real degradation is noisier, sensors are less cleanly related, and failures are rarer and less labelled.
- **Few failure events.** 37 failures across 20 machines is a small sample; test-set metrics rest on a handful of events and carry wide uncertainty.
- **Near-perfect scores are expected here** and should not be quoted as an achievement. On real data, a good result would be far lower and would need careful threshold and cost analysis.
- **MongoDB was not executed.** The module is written to the documented API but untested; SQLite was the executed storage layer.
- **The Power BI file (.pbix) is not included.** It can only be authored in Power BI Desktop; this submission provides the data model, measures and step-by-step guide, and a Matplotlib rendering of the dashboard.
- **Correlation pruning at 0.98** is a judgement call and reflects this dataset's structure.
- **Offline detection depends on the maintenance log.** If downtime were not logged, offline hours would be treated as telemetry loss and imputed, which would be wrong.

11.2 Future work

- Run the identical pipeline on a public real dataset (for example NASA C-MAPSS turbofan degradation data or the AI4I 2020 predictive-maintenance dataset) and compare behaviour; the configuration block makes this a small change.
- Stream the pipeline: replace batch reads with incremental ingestion and rolling-window feature computation.
- Add automated data-quality monitoring (drift in sensor distributions, rising missing rates) with alerts.

- Add cost-sensitive evaluation (the cost of a missed failure versus a false alarm) and probability calibration.
- Add concept-drift handling and periodic retraining; store feature definitions in a feature store for training/serving consistency.
- Extend to survival models that treat censoring explicitly instead of capping RUL.

12. Reproducibility: Project Structure and How to Run

```
project/
  data/raw/         sensor_readings.csv, maintenance_logs.json, machine_master.csv
  data/processed/   cleaned_sensor_data.csv
                    ai_ready_dataset.csv, ai_ready_dataset_scaled.csv
  data/warehouse.db SQLite warehouse
  src/              generate_data.py, pipeline.py, model_check.py,
                    dashboard.py, mongo_store.py, build_report.py
  reports/          pipeline_stats.json, imputation_evaluation.csv,
                    feature_importance.csv, feature_catalog.csv,
                    model_baseline.json, scaler_params.json, pipeline.log
  figures/          dashboard pages and report figures
  powerbi/          star-schema CSVs for Power BI
```

Requirements: Python 3.10+, pandas, numpy, scikit-learn, scipy, matplotlib, python-docx (and pymongo only for the optional MongoDB step). Run in this order:

```
python src/generate_data.py   # simulated raw sources (skip for real data)
python src/pipeline.py        # ingest, clean, impute, features (~20 s)
python src/model_check.py     # baseline models (~2.5 min)
python src/dashboard.py       # dashboard pages + Power BI exports
python src/build_report.py    # this document
```

To use real data, place files with the same columns in `data/raw/` (or edit the ingest function) and adjust the `SENSORS`, `LIMITS` and split dates in the configuration block at the top of `pipeline.py`. Every run writes `reports/pipeline.log` and `reports/pipeline_stats.json`, so each number in this document can be traced to a run.

13. Conclusion and Learning Outcomes

The project delivers a complete, reproducible route from messy multi-source machine data to a validated, documented, model-ready dataset, together with evidence that each step helped: defects were found at about the rates they were injected, imputation was chosen by measured error, and engineered features improved precision and RUL accuracy over raw sensors. Its main lesson is that most of the effort and most of the risk in predictive maintenance sit before the model, in the data.

Table 16. Mapping to the stated learning outcomes

Learning outcome	Demonstrated by
Data Ingestion	Three heterogeneous sources, explicit typing, three timestamp formats, regular grid, storage layer (Sec. 5)
Data Cleaning	Eight ordered validation rules with counted effects and post-clean assertions (Sec. 6)
Feature Engineering	79 candidate features in 9 families, leakage controls, correlation pruning, importance analysis (Sec. 8)
AI Data Preparation	Targets, censoring treatment, time-based split, train-only scaling, baseline-model validation (Sec. 9)

Appendix A. Feature Catalogue (data dictionary)

All candidate features. 'Kept' features are in the AI-ready dataset; 'Pruned' features were removed for redundancy. Importance is the random-forest importance (kept features only).

Table 17. Feature catalogue

Feature	Family	Description	Status
temperature	Raw sensor values	Cleaned hourly temperature (deg C)	Kept
vibration	Raw sensor values	Cleaned hourly vibration (mm/s RMS)	Kept
pressure	Raw sensor values	Cleaned hourly pressure (bar)	Kept
rpm	Raw sensor values	Cleaned hourly rpm (rev/min)	Kept
current	Raw sensor values	Cleaned hourly current (A)	Kept
oil_quality	Raw sensor values	Cleaned hourly oil quality (index 0-100)	Kept
temperature_mean_6h	Rolling statistics	6 h rolling mean of temperature	Kept
temperature_std_6h	Rolling statistics	6 h rolling std. dev. of temperature	Kept
temperature_mean_24h	Rolling statistics	24 h rolling mean of temperature	Kept
temperature_std_24h	Rolling statistics	24 h rolling std. dev. of temperature	Kept
temperature_max_24h	Rolling statistics	24 h rolling maximum of temperature	Kept
temperature_delta_1h	Rate of change	temperature: change versus 1 h earlier	Kept
temperature_delta_6h	Rate of change	temperature: change versus 6 h earlier	Kept
vibration_std_6h	Rolling statistics	6 h rolling std. dev. of vibration	Kept
vibration_std_24h	Rolling statistics	24 h rolling std. dev. of vibration	Kept
vibration_max_24h	Rolling statistics	24 h rolling maximum of vibration	Kept
vibration_delta_1h	Rate of change	vibration: change versus 1 h earlier	Kept
vibration_delta_6h	Rate of change	vibration: change versus 6 h earlier	Kept
pressure_mean_6h	Rolling statistics	6 h rolling mean of pressure	Kept
pressure_std_6h	Rolling statistics	6 h rolling std. dev. of pressure	Kept
pressure_std_24h	Rolling statistics	24 h rolling std. dev. of pressure	Kept
pressure_max_24h	Rolling statistics	24 h rolling maximum of pressure	Kept
pressure_min_24h	Rolling statistics	24 h rolling minimum of pressure	Kept
pressure_delta_1h	Rate of change	pressure: change versus 1 h earlier	Kept
pressure_delta_6h	Rate of change	pressure: change versus 6 h earlier	Kept
rpm_std_6h	Rolling statistics	6 h rolling std. dev. of rpm	Kept
rpm_std_24h	Rolling statistics	24 h rolling std. dev. of rpm	Kept
rpm_delta_1h	Rate of change	rpm: change versus 1 h earlier	Kept
rpm_delta_6h	Rate of change	rpm: change versus 6 h earlier	Kept
current_mean_6h	Rolling statistics	6 h rolling mean of current	Kept
current_std_6h	Rolling statistics	6 h rolling std. dev. of current	Kept
current_std_24h	Rolling statistics	24 h rolling std. dev. of current	Kept
current_max_24h	Rolling statistics	24 h rolling maximum of current	Kept
current_min_24h	Rolling statistics	24 h rolling minimum of current	Kept
current_delta_1h	Rate of change	current: change versus 1 h earlier	Kept
current_delta_6h	Rate of change	current: change versus 6 h earlier	Kept
oil_quality_std_6h	Rolling statistics	6 h rolling std. dev. of oil quality	Kept
oil_quality_std_24h	Rolling statistics	24 h rolling std. dev. of oil quality	Kept
oil_quality_min_24h	Rolling statistics	24 h rolling minimum of oil quality	Kept
oil_quality_delta_1h	Rate of change	oil quality: change versus 1 h earlier	Kept
oil_quality_delta_6h	Rate of change	oil quality: change versus 6 h earlier	Kept
temperature_slope_24h	Trend	24 h linear trend slope of temperature (per hour)	Kept
vibration_slope_24h	Trend	24 h linear trend slope of vibration (per hour)	Kept
oil_quality_slope_24h	Trend	24 h linear trend slope of oil quality (per hour)	Kept
vibration_z_baseline	Baseline deviation	6 h mean of vibration as robust z-score vs own healthy baseline	Kept
rpm_z_baseline	Baseline deviation	6 h mean of rpm as robust z-score vs own healthy baseline	Kept
oil_quality_z_baseline	Baseline deviation	6 h mean of oil quality as robust z-score vs own healthy baseline	Kept
rpm_ratio	Asset context	6 h mean rpm divided by rated rpm	Kept
rated_power_kw	Asset context	Rated power of the asset (kW)	Kept
machine_age_days	Asset context	Days since installation	Kept
hour_sin	Time of day / week	Hour of day, sine encoding	Kept
hour_cos	Time of day / week	Hour of day, cosine encoding	Kept
day_of_week	Time of day / week	Day of week (0 = Monday)	Kept
hours_since_last_maint	Maintenance history	Hours since the last maintenance event (past events only)	Kept
failures_to_date	Maintenance history	Number of failures recorded so far	Kept
type_pump	Asset context	1 if machine type is pump	Kept
type_compressor	Asset context	1 if machine type is compressor	Kept
type_motor	Asset context	1 if machine type is motor	Kept
type_turbine	Asset context	1 if machine type is turbine	Kept

Feature	Family	Description	Status
temperature_min_24h	Rolling statistics	24 h rolling minimum of temperature	Pruned (r=0.982 with temperature_mean_24h)
vibration_mean_6h	Rolling statistics	6 h rolling mean of vibration	Pruned (r=0.985 with temperature_mean_24h)
vibration_mean_24h	Rolling statistics	24 h rolling mean of vibration	Pruned (r=0.994 with temperature_mean_24h)
vibration_min_24h	Rolling statistics	24 h rolling minimum of vibration	Pruned (r=0.982 with temperature_mean_24h)
pressure_mean_24h	Rolling statistics	24 h rolling mean of pressure	Pruned (r=0.991 with temperature_mean_24h)
rpm_mean_6h	Rolling statistics	6 h rolling mean of rpm	Pruned (r=1.000 with rpm)
rpm_mean_24h	Rolling statistics	24 h rolling mean of rpm	Pruned (r=1.000 with rpm)
rpm_max_24h	Rolling statistics	24 h rolling maximum of rpm	Pruned (r=0.999 with rpm)
rpm_min_24h	Rolling statistics	24 h rolling minimum of rpm	Pruned (r=0.999 with rpm)
current_mean_24h	Rolling statistics	24 h rolling mean of current	Pruned (r=0.983 with temperature_mean_24h)
oil_quality_mean_6h	Rolling statistics	6 h rolling mean of oil quality	Pruned (r=0.990 with oil_quality)
oil_quality_mean_24h	Rolling statistics	24 h rolling mean of oil quality	Pruned (r=0.986 with oil_quality)
oil_quality_max_24h	Rolling statistics	24 h rolling maximum of oil quality	Pruned (r=0.983 with oil_quality)
temperature_ewma_12h	Trend	12 h exponentially weighted mean of temperature	Pruned (r=0.993 with temperature_mean_6h)
vibration_ewma_12h	Trend	12 h exponentially weighted mean of vibration	Pruned (r=0.990 with temperature_mean_24h)
temperature_z_baseline	Baseline deviation	6 h mean of temperature as robust z-score vs own healthy baseline	Pruned (r=0.992 with temperature_mean_6h)
pressure_z_baseline	Baseline deviation	6 h mean of pressure as robust z-score vs own healthy baseline	Pruned (r=0.992 with pressure_mean_6h)
current_z_baseline	Baseline deviation	6 h mean of current as robust z-score vs own healthy baseline	Pruned (r=0.997 with current_mean_6h)
vib_per_amp	Cross-sensor ratios	Vibration (6 h mean) divided by current (6 h mean)	Pruned (r=0.987 with temperature_mean_24h)
temp_pressure_ratio	Cross-sensor ratios	Temperature (6 h mean) divided by pressure (6 h mean)	Pruned (r=0.991 with temperature_mean_24h)

Appendix B. Power BI Build Guide and DAX Measures

Power BI Desktop (Windows) is required to assemble the interactive report. The folder `powerbi/` contains the model tables; the steps below take about 30 to 45 minutes.

B.1 Load the data

1. Home > Get data > Text/CSV, and load: `dim_machine.csv`, `fact_sensor_daily.csv`, `fact_sensor_hourly.csv`, `fact_maintenance.csv`, `data_quality_scorecard.csv`, `data_quality_issues.csv`, `feature_importance.csv`.
2. In Power Query, confirm `date` and `timestamp` are Date / Date-Time types and `event_ts` is Date-Time.
3. Create a date table: Modeling > New table > `DimDate = CALENDARAUTO()`.

B.2 Model (star schema)

Table 18. Relationships

From (one side)	To (many side)	Cardinality
dim_machine[machine_id]	fact_sensor_daily[machine_id]	1 : *
dim_machine[machine_id]	fact_sensor_hourly[machine_id]	1 : *
dim_machine[machine_id]	fact_maintenance[machine_id]	1 : *
DimDate[Date]	fact_sensor_daily[date]	1 : *

B.3 DAX measures

```
Avg Vibration = AVERAGE ( fact_sensor_daily[avg_vibration] )
Max Vibration = MAX ( fact_sensor_daily[max_vibration] )
Avg Temperature = AVERAGE ( fact_sensor_daily[avg_temperature] )

Failure Count =
  CALCULATE ( COUNTROWS ( fact_maintenance ),
              fact_maintenance[event_type] = "FAILURE" )

Downtime Hours =
  CALCULATE ( SUM ( fact_maintenance[downtime_hours] ),
              fact_maintenance[event_type] = "FAILURE" )

Failure Cost (INR) =
  CALCULATE ( SUM ( fact_maintenance[cost_inr] ),
              fact_maintenance[event_type] = "FAILURE" )

Operating Hours = SUM ( fact_sensor_daily[readings] )
MTBF (hours) = DIVIDE ( [Operating Hours], [Failure Count] )

Imputed Share % =
  DIVIDE ( SUM ( fact_sensor_daily[imputed_cells] ),
           SUM ( fact_sensor_daily[readings] ) * 6 )

Machines At Risk =
  CALCULATE ( DISTINCTCOUNT ( fact_sensor_daily[machine_id] ),
              fact_sensor_daily[failure_risk_window] = 1 )
```

B.4 Report pages

Table 19. Suggested Power BI pages

Page	Visuals

Page	Visuals
1. Overview	Cards: Failure Count, Downtime Hours, Failure Cost, MTBF, Machines At Risk. Slicers: machine type, location, date.
2. Data quality	Clustered bar from data_quality_scorecard (raw vs clean); bar chart from data_quality_issues; card for Imputed Share %.
3. Machine health	Line chart: Avg Vibration and Avg Temperature by date with a machine slicer; matrix machine x week with conditional-format background on Avg Vibration (the fleet heatmap).
4. Failure analysis	Column chart: Failure Count by failure_mode; bar: Failure Cost by machine; table of events.
5. AI readiness	Bar chart from feature_importance (top 15); card with row and feature counts.

The two dashboard images in Section 10 show the intended layout; the numbers in them come from the same tables you load here.